

Anggota Kelompok: Aghniya Afiatul Jannah (2306035)
Kailla Salsabila (2306064)

Laporan Dokumentasi Teknis Proyek: My Movie List

1. Pendahuluan

1.1 Latar Belakang dan Tujuan Proyek

My Movie List adalah aplikasi mobile berbasis Android/iOS yang dikembangkan menggunakan framework Flutter. Proyek ini bertujuan untuk menyediakan *platform* interaktif bagi pengguna agar dapat menjelajahi basis data film secara *real-time* (terintegrasi dengan TMDB API). Pengguna tidak hanya bisa melihat film yang sedang tren, namun juga bisa menyimpan film ke dalam daftar personal (Wishlist), menandai film yang sudah ditonton (Watched), serta memberikan ulasan berbasis bintang (Rating & Review) yang tersimpan secara terpusat di dalam database cloud.

1.2 Lingkungan Teknologi (Tech Stack)

Aplikasi ini dibangun menggunakan arsitektur *frontend-heavy* dengan integrasi layanan *Backend-as-a-Service* (BaaS) dan RESTful API eksternal.

- Bahasa Pemrograman Utama: Dart (versi stabil terbaru).
- Framework UI: Flutter (Multi-platform).
- Sistem Autentikasi: Firebase Authentication (Otentikasi Berbasis Email & Password).
- Sistem Database: Firebase Cloud Firestore (NoSQL Document-Based Database).
- Sumber Data (REST API): TMDB (The Movie Database) API versi 3.
- Layanan Penyimpanan Gambar: ImgBB API (untuk konversi aset lokal menjadi URL publik tanpa menyita kuota Firebase Storage).

2. Arsitektur dan Struktur Proyek

2.1 Arsitektur Aplikasi

Aplikasi mengusung pendekatan Monolithic Stateful Architecture. Pembaruan antarmuka (UI) dikendalikan melalui hierarki StatefulWidget dengan pengelolaan status lokal (setState()). Tidak ada *library* manajemen state tambahan seperti BLoC atau Provider yang digunakan, sehingga data mengalir dari orangtua ke anak (*prop-drilling*) dan dikelola langsung pada siklus hidup halaman (*State Lifecycle*).

2.2 Struktur Direktori (/lib)

Struktur folder membagi logika secara modular untuk memisahkan UI dan koneksi layanan backend:

```
lib/
├── core/
│   └── constants.dart    # Menyimpan konstanta warna utama (AppColors) & kredensial API
│                           (AppConstants)
├── models/
│   └── movie_model.dart  # Representasi Data Class (Model) untuk data film dari TMDB
├── screens/
│   ├── auth/
│   │   ├── login_screen.dart  # Antarmuka login pengguna
│   │   └── register_screen.dart # Antarmuka pendaftaran akun baru
│   ├── detail_screen.dart    # Antarmuka detail film lengkap (Trailer, Sinopsis, Aksi)
│   └── genre_movies_screen.dart # Antarmuka film berdasarkan filter genre
```

```

— help_center_screen.dart # Antarmuka pusat bantuan / FAQ pengguna
— home_screen.dart      # Antarmuka dashboard (Trending, Popular, Upcoming)
— main_nav.dart         # Navigasi utama aplikasi (BottomNavigationBar)
— my_list_screen.dart    # Antarmuka daftar Wishlist dan Watched milik pengguna
— profile_screen.dart    # Antarmuka manajemen akun dan statistik
— review_history_screen.dart # Antarmuka riwayat ulasan film
— search_screen.dart     # Antarmuka pencarian film & list genre
— splash_screen.dart     # Antarmuka inialisasi aplikasi (Delay & Auth Check)
— trending_screen.dart   # Antarmuka daftar film yang sedang tren lengkap (Pagination)
— upcoming_screen.dart   # Antarmuka daftar film akan datang (Pagination)
— services/
  — api_service.dart     # Kelas abstraksi HTTP Request ke TMDB API
  — firebase_service.dart # Kelas abstraksi CRUD Firestore dan Firebase Auth
  — imgbb_service.dart   # Kelas abstraksi HTTP POST ke ImgBB API untuk unggah
gambar
— firebase_options.dart  # Auto-generated file konfigurasi koneksi Firebase
— main.dart              # Titik awal eksekusi aplikasi (Entry Point)

```

2.3 Manajemen Dependensi (pubspec.yaml)

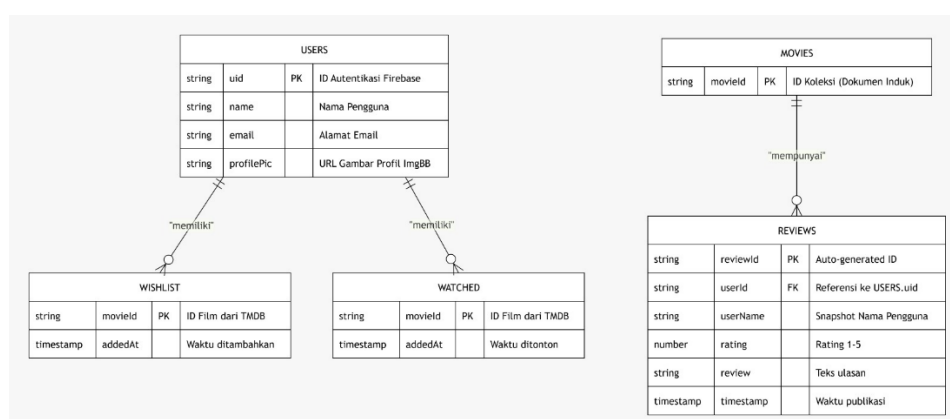
Semua fungsionalitas pihak ketiga dideklarasikan di sini:

- flutter/material: Library desain antarmuka bawaan Flutter.
- firebase_core: Modul dasar inialisasi ekosistem Firebase.
- firebase_auth: Menangani logika pendaftaran, login, logout, dan manajemen sesi kredensial.
- cloud_firestore: Melakukan interaksi baca/tulis (*read/write*) dokumen NoSQL secara real-time.
- http: Memungkinkan aplikasi mengirim HTTP GET dan POST request ke server pihak ketiga (TMDB & ImgBB).
- image_picker: Mengakses kamera atau galeri sistem operasi host untuk mengambil foto (digunakan pada fitur ubah foto profil).
- url_launcher: Menjembatani aplikasi untuk membuka URL (link trailer YouTube) di aplikasi browser atau aplikasi YouTube perangkat.

3. Desain Struktur Data dan Basis Data

3.1 Skema Entity-Relationship (Firebase Cloud Firestore)

Firestore bersifat NoSQL, sehingga relasi data dibentuk menggunakan struktur *Collection*, *Document*, dan *Sub-Collection*.



3.2 Penjelasan Entitas Database

1. **Koleksi Utama users:** Digunakan untuk menyimpan profil pengguna. ID dokumen menggunakan UID yang dihasilkan secara otomatis oleh `firebase_auth` ketika pengguna berhasil register.
 - Sub-koleksi wishlist: Daftar ID film yang ingin ditonton. Hanya menyimpan parameter waktu untuk optimasi *query*. Data film aslinya diambil secara asinkron dari TMDB API.
 - Sub-koleksi watched: Daftar ID film yang sudah ditonton. Struktur serupa dengan wishlist.
2. **Koleksi Utama movies:** Digunakan khusus untuk menyimpan interaksi sosial pengguna terhadap film.
 - Sub-koleksi reviews: Setiap kali ada pengguna memberikan ulasan pada suatu film, akan dibuatkan dokumen ber-ID acak di bawah sub-koleksi ini. Data ini digabungkan secara manual dengan ID film agar bisa ditampilkan pada halaman *Review History*.

3.3 Dokumentasi Endpoint API Eksternal

Aplikasi mengkonsumsi REST API tanpa backend sendiri.

A. TMDB API (Base URL: <https://api.themoviedb.org/3>)

- Parameter wajib: `api_key` & `language=id-ID`.
- GET `/trending/movie/week`: Menarik daftar film dengan *engagement* tertinggi di dunia minggu ini. Membutuhkan parameter `page` untuk *infinite scroll*.
- GET `/movie/popular`: Menarik daftar film dengan rating/popularitas kumulatif tertinggi.
- GET `/movie/upcoming`: Menarik daftar film yang akan dirilis di bioskop. Membutuhkan parameter `page`.
- GET `/search/movie?query={teks}`: Endpoint *full-text search* untuk menelusuri database film TMDB. Membutuhkan parameter `page`.
- GET `/movie/{id}`: Menarik spesifikasi teknis film (durasi/runtime, daftar genre secara array).
- GET `/movie/{id}?language=en-US`: (FALLBACK) Digunakan khusus jika data overview (sinopsis) pada request bahasa Indonesia menghasilkan *string* kosong.
- GET `/movie/{id}/credits`: Mengembalikan array cast (aktor). Aplikasi melakukan `.take(10)` untuk hanya menampilkan 10 aktor teratas demi efisiensi render memori UI.
- GET `/movie/{id}/videos`: Mengembalikan array video promosi. Logika aplikasi menyaring video dengan atribut `site == "YouTube"` dan `type == "Trailer"` untuk diekstraksi *key* videonya.
- GET `/discover/movie?with_genres={id}`: Mengembalikan film berdasarkan tag genre. Membutuhkan parameter `page`.
- GET `/genre/movie/list`: Mengembalikan referensi statis daftar ID genre dan namanya.

B. ImgBB API (Base URL: <https://api.imgbb.com/1>)

- POST `/upload?key={api_key}`: Endpoint penerima *multipart/form-data*. Gambar dari perangkat dibaca sebagai Bytes (kIsWeb) atau Path (Android), lalu dikirim ke server. Respons sukses mengembalikan JSON berisi `data.display_url` yang kemudian disimpan ke kolom `profilePic` di Firestore.

4.1 Flowchart Autentikasi (Login, Register, dan Cek Sesi)

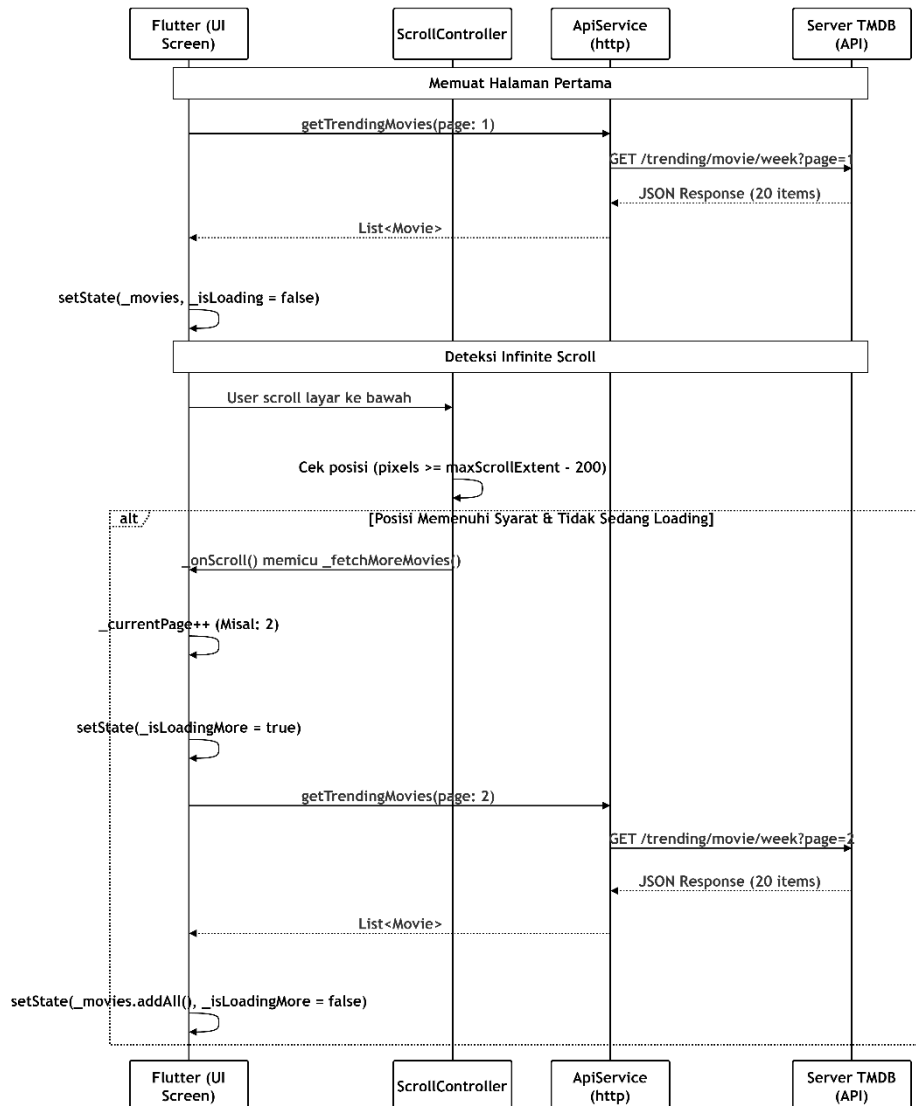
Alur ini mencakup bagaimana aplikasi memeriksa sesi pengguna saat baru dibuka (Splash Screen), serta proses pendaftaran (Register) dan masuk (Login).



- Tujuan: Memastikan hanya pengguna terverifikasi yang dapat mengakses aplikasi utama.
- Implementasi Source Code: Di `splash_screen.dart`, delay 3 detik dilakukan sebelum mengecek `currentUser`. Register mengirimkan data Auth dan sekaligus membuat dokumen kosong di Firestore (`_db.collection('users').doc(uid).set()`) sebagai penampung data profil kelak. Logout memanggil `FirebaseAuth.instance.signOut()` dan mereset tumpukan navigasi kembali ke `LoginScreen`.

4.2 Alur Interaksi Klien & API Eksternal (Pagination)

Alur ini menggambarkan bagaimana aplikasi memuat daftar film dari TMDB API, termasuk mekanisme *Infinite Scroll* yang ada pada halaman Trending, Upcoming, Search, dan Genre.

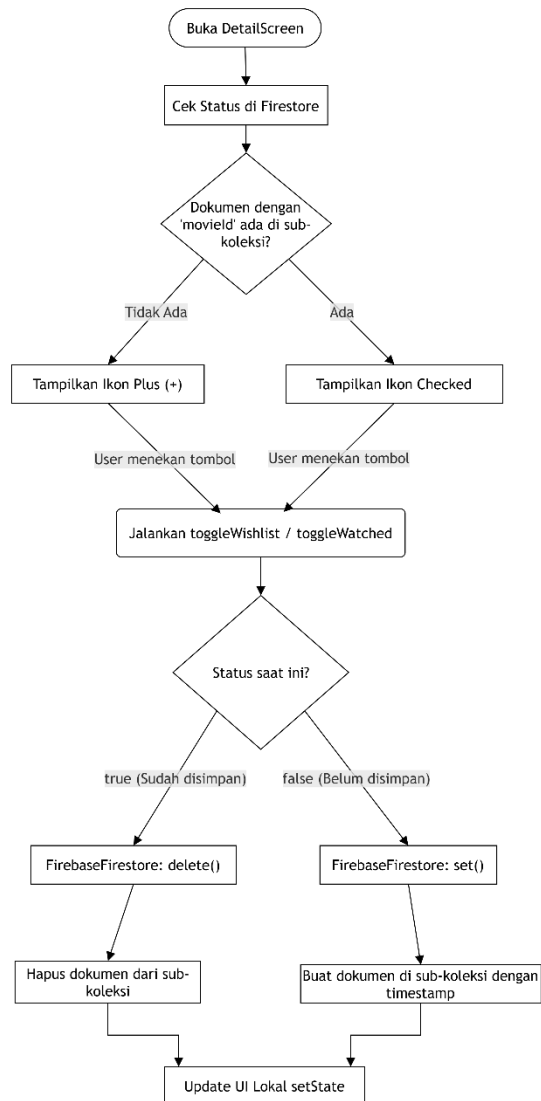


Penjelasan Alur:

- Tujuan: Mengelola pergerakan data bervolume besar secara bertahap tanpa membekukan layar (UI).
- Implementasi Source Code: Setiap layar daftar film diikat dengan `ScrollController.addListener(_onScroll)`. Jika sisa jarak *scroll* kurang dari 200 piksel, fungsi mengirim HTTP GET via ApiService dengan parameter `&page=` dinamis.

4.3 Flowchart Interaksi Film (Wishlist & Watched)

Alur ketika pengguna menambahkan suatu film ke daftar *Wishlist* atau *Watched* melalui halaman DetailScreen.

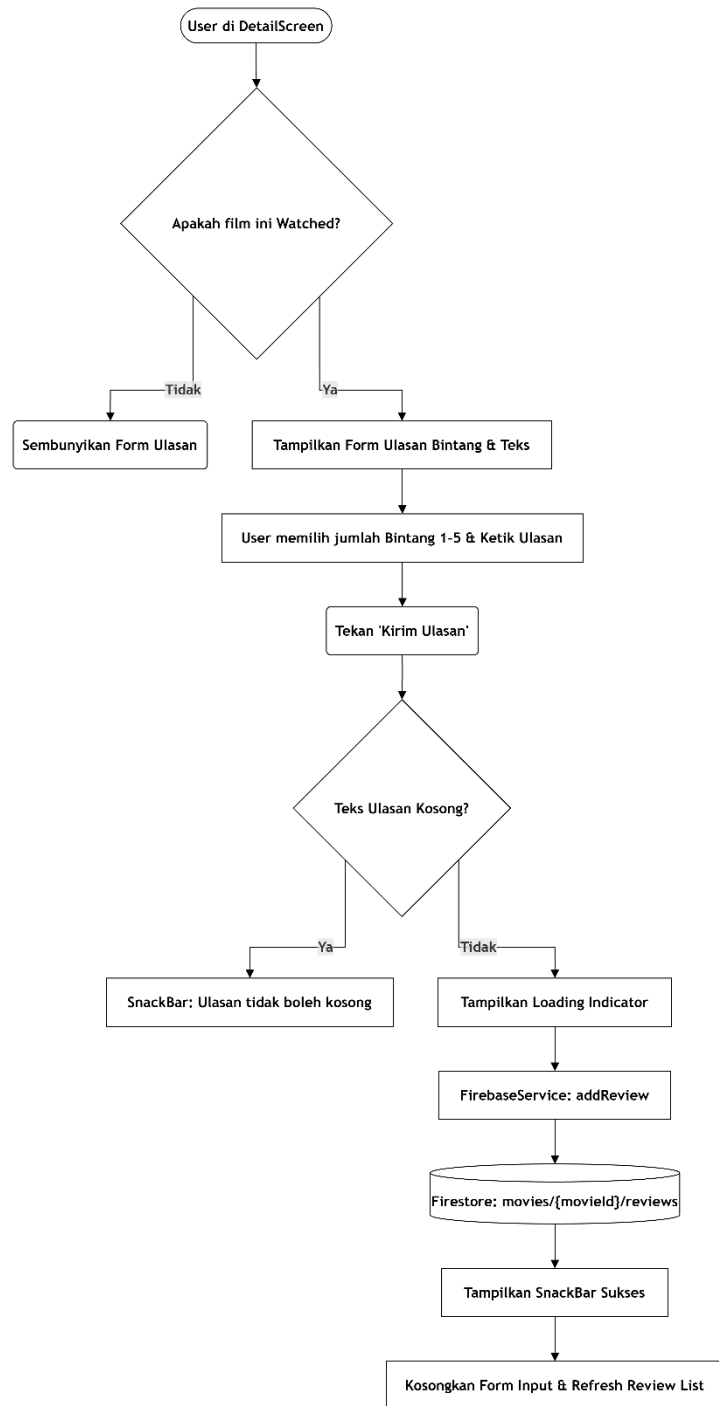


Penjelasan Alur:

- Tujuan: Menyimpan data referensi film (*bookmark*) pengguna ke database cloud secara instan (*optimistic update*).
- Implementasi Source Code: Dikelola oleh `FirestoreService`. Logika menggunakan rujukan koleksi: `users/{uid}/wishlist/{movieId}`. Data disimpan berupa timestamp.

4.4 Flowchart Fitur Pembuatan Ulasan (Review)

Menggambarkan alur proses memberikan rating bintang dan ulasan teks yang diikat pada ID Film.

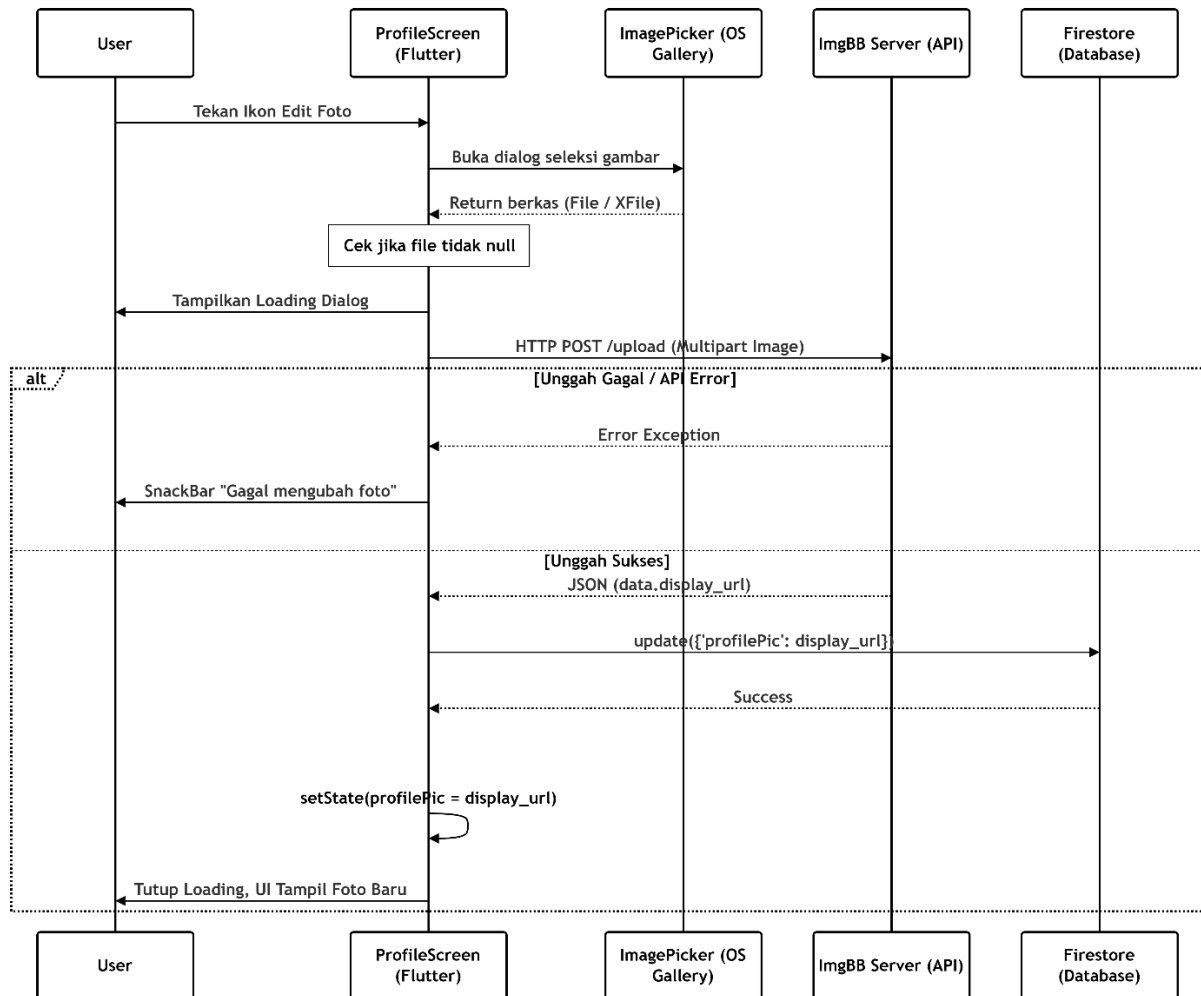


Penjelasan Alur:

- Tujuan: Mengumpulkan interaksi (*User Generated Content*) terikat pada satu entitas film (movieId).
- Implementasi Source Code: Form dirender jika `_isWatched` bernilai `true`. Data di-set ke koleksi global (movies/{movieId}/reviews).

4.5 Diagram Pengubahan Foto Profil (Integrasi ImgBB)

Alur manajemen file lokal yang diubah menjadi URL publik.

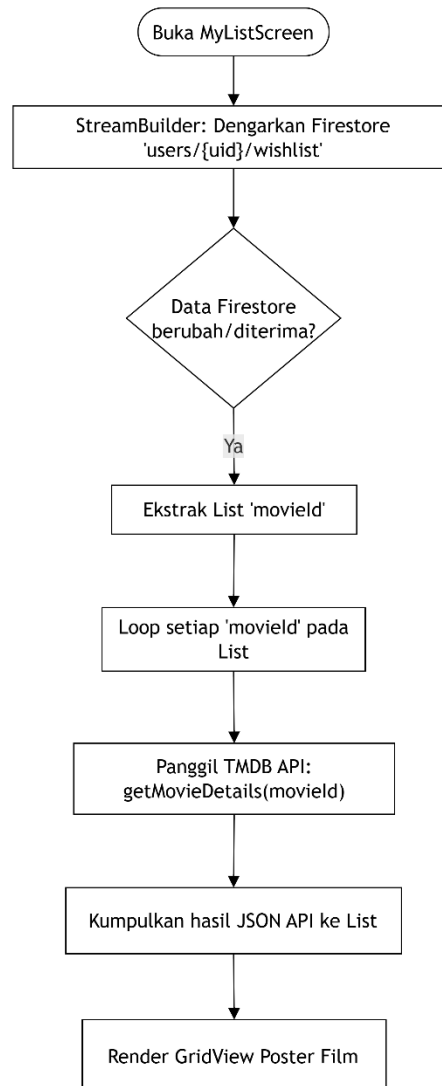


Penjelasan Alur:

- Tujuan: Menghindari beban penyimpanan blob database dengan merubah aset menjadi URL string.
- Implementasi Source Code: Menggunakan `imgbb_service.dart`. Respon HTTP disisipkan ke Firestore via `FirestoreService.updateProfilePic()`.

4.6 Flowchart Sinkronisasi Data 'My List'

Alur pembacaan data campuran antara *State* Firebase dan TMDB API.



Penjelasan Alur:

- Tujuan: Mempertahankan kebaruan data tanpa perlu menyimpan JSON TMDB di dalam Firestore.
- Implementasi Source Code: `_fetchMoviesDetails()` menerima *List* dari Stream Firestore, lalu mengeksekusi request `getMovieDetails(id)` ke TMDB API secara massal.

5. Dokumentasi Terperinci Berdasarkan Halaman (UI/UX)

5.1 Splash Screen

- Fungsi Halaman: Sebagai jembatan visual pertama sebelum proses asinkron pengecekan otentikasi selesai dievaluasi.
- Komponen Kunci: Widget `Image.asset` berukuran 300x300 dengan penanganan kesalahan `errorBuilder`.
- Validasi Kritis: Menggunakan kondisional `if (!mounted) return;` pasca fungsi asinkron `Future.delayed` agar aplikasi tidak mengalami kebingungan alur *context* apabila aplikasi tiba-tiba diminimalkan pengguna di tengah jalan.

- **Screenshot Halaman Splash**

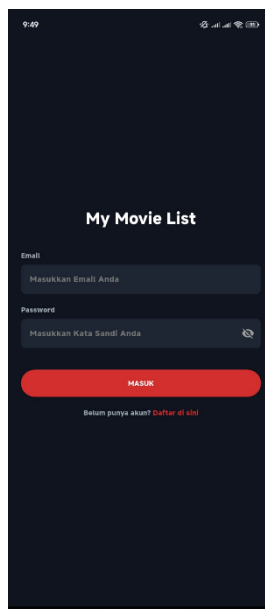


Menampilkan sebuah logo bundar bertema film dengan tulisan My Movie List di tengah layar berlatar belakang biru-abu gelap (0xFF0F141E).

5.2 Modul Autentikasi (Login & Register Screen)

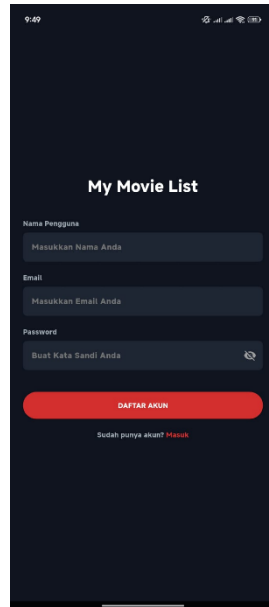
- Fungsi Halaman: Pintu masuk keamanan pengguna.
- Komponen Kunci: Kumpulan TextFormField dan logika asinkron pemanggilan `FirebaseAuth.instance.signInWithEmailAndPassword` serta `createUserWithEmailAndPassword`.
- Logika Khusus Register: Selain membuat kredensial, register juga akan mengirimkan instruksi `_db.collection('users').doc(user.uid).set()` untuk menginisialisasi entri profil kosong dalam format JSON di Firestore.

- **Screenshot Halaman Login**



Menampilkan form input Email dan Password, tombol Sign In berwarna merah bata, serta teks link menuju halaman pendaftaran di bawah.

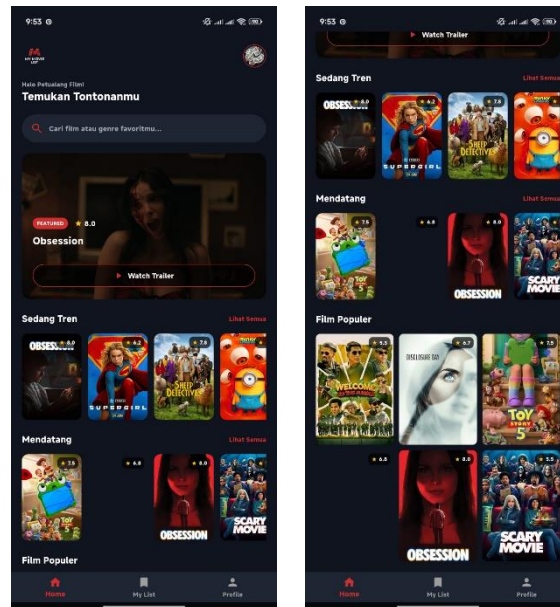
- **Screenshot Halaman Register**



Menampilkan form input Nama Lengkap, Email, dan Password (dengan obscureText aktif), serta tombol Sign Up.

5.3 Home Screen (Dashboard Utama)

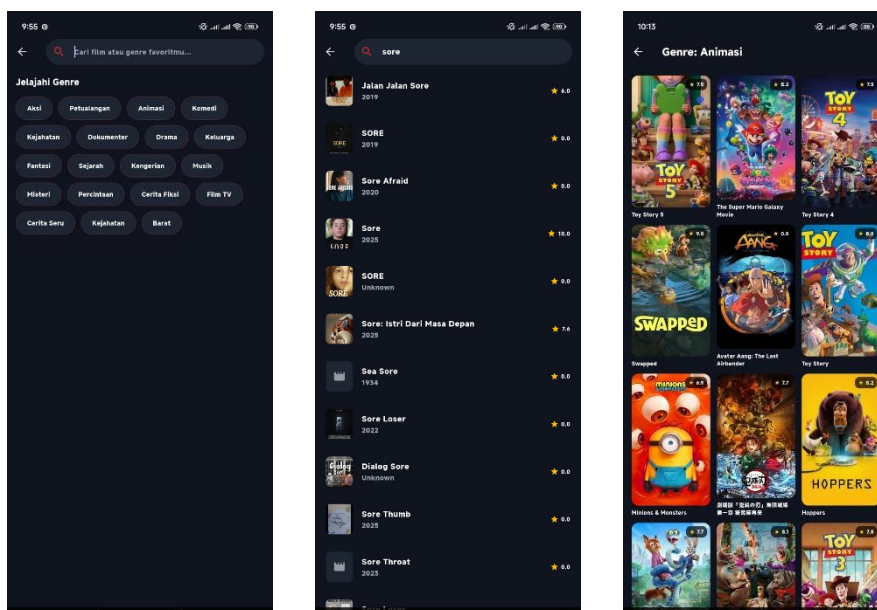
- Fungsi Halaman: Menampilkan ikhtisar kurasi katalog TMDB.
- Alur Kerja: Menggunakan komponen FutureBuilder yang bersarang. Apabila `snapshot.connectionState == ConnectionState.waiting`, indikator `CircularProgressIndicator` dirender.
- Komponen UI Khusus:
 - Banner Carousel: Merender poster dengan `BoxFit.cover` dipotong membulat dengan `ClipRRect`. Menggunakan bayangan gradien transparan (*ShaderMask*) untuk visibilitas teks di atas gambar.
 - Daftar Horizontal (Trending & Populer): Render poster menyamping menggunakan `ListView.builder(scrollDirection: Axis.horizontal)`.
- **Screenshot Dashboard/Home**



Menampilkan slider banner di bagian atas (sedang tren), deretan daftar populer horizontal, dan deretan rilis mendatang.

5.4 Fitur Ekosistem Film (Pencarian & Penelusuran Genre)

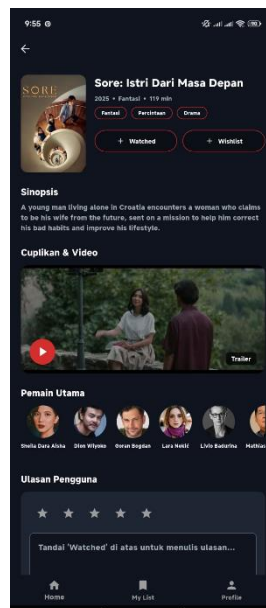
- Fungsi Halaman: Halaman mandiri tempat pengguna memasukkan kata kunci penelusuran.
- Alur Data: Input pengguna dimasukkan pada event `onSubmitted` -> mereset `page = 1` -> Memanggil fungsi `searchMovies(query, page)`.
- Logika UI Cerdas: Saat *search bar* dikosongkan (`onChanged`), layar kembali menampilkan *Grid Wrap* berupa tombol kotak kategori genre.
- **Screenshot Halaman Pencarian**



Menampilkan AppBar berupa kotak pencarian. Bagian badan menampilkan grid "Jelajahi Genre" apabila belum mencari, atau daftar list memanjang ke bawah berisikan poster film kecil, tahun rilis, dan tombol bintang rating apabila sedang mencari.

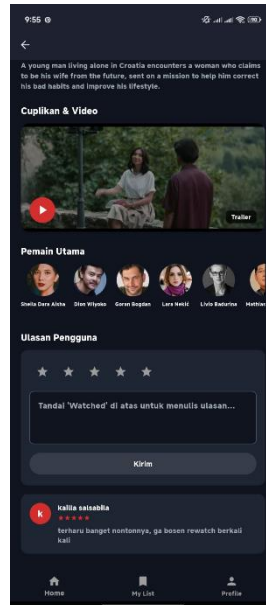
5.5 Halaman Detail Film (Krustial & Terkompleks)

- Fungsi Halaman: Memberikan gambaran komprehensif, titik aksi penyimpanan pengguna, serta jembatan interaksi komunitas.
- Sistem *State*:
 - `_isWatched` dan `_isWishlist` terus-menerus disinkronisasi ke dokumen spesifik Firestore milik pengguna. Menekan tombol akan memicu penambahan atau penghapusan dokumen secara instan (*optimistic UI rendering*).
- Pemrosesan Trailer: Data balikan TMDB berupa YouTube Video Key disambung string URL `https://www.youtube.com/watch?v={key}`, lalu di-*trigger* ke peramban menggunakan pustaka eksternal `url_launcher`.
- Kondisional UI: Poster akan menggunakan logo cadangan jika atribut `posterPath` dari API mereturn nilai string kosong. Rating menggunakan `.toStringAsFixed(1)` untuk menjamin format nilai tidak berlebihan.
- Screenshot Halaman Detail (Atas)



Menampilkan poster film besar menutupi separuh atas layar, disusul judul tebal, tahun, durasi, tag genre, serta 2 tombol aksi (" + Watched" dan " + Wishlist") berdampingan.

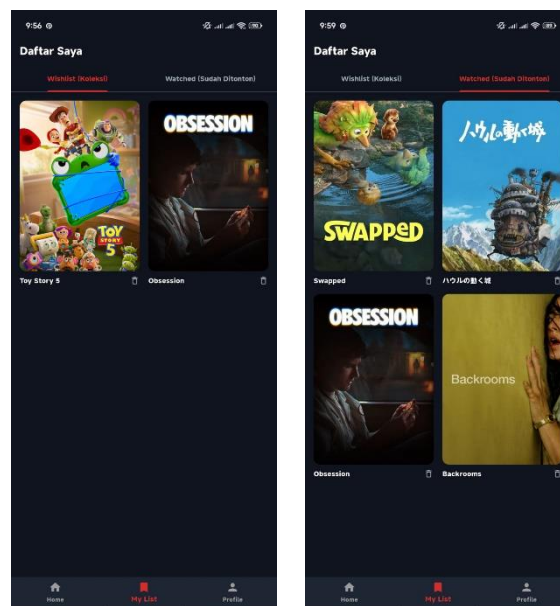
- **Screenshot Halaman Detail (Bawah - Review Area)**



Menampilkan gambar thumbnail YouTube, jejeran foto bundar aktor pemeran utama (Cast), serta kotak input teks beserta pemilihan 5 ikon bintang dan tombol 'Kirim Ulasan'.

5.6 Halaman Koleksi Saya (My List Screen)

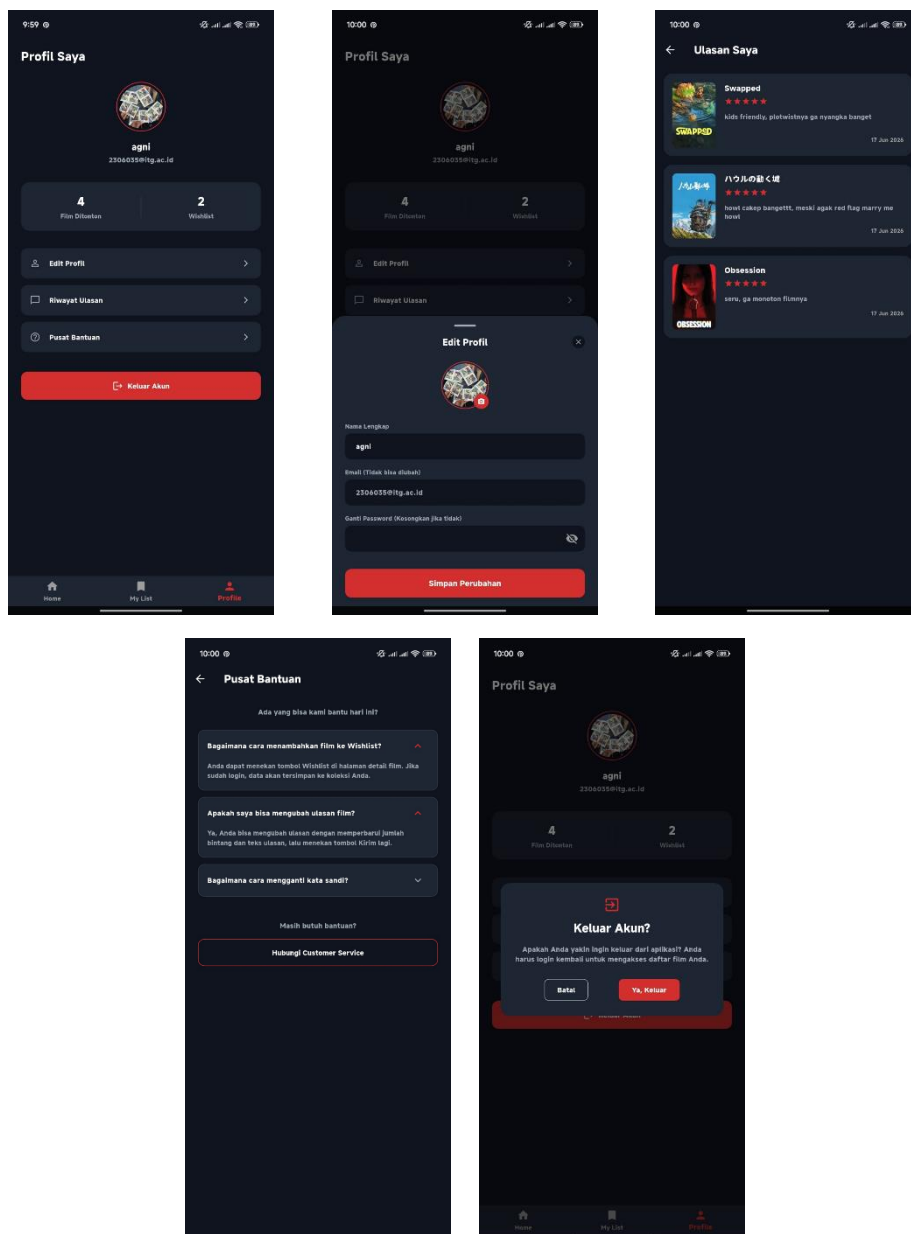
- Fungsi Halaman: Jembatan komunikasi antara data yang dipegang Firestore dan UI pengguna.
- Sistem Sinkronisasi Berantai (Data Piping):
 - Memakai widget DefaultTabController dan tab view.
 - Alur: UI membungkus layar dengan StreamBuilder ke rujukan users/{uid}/wishlist. Setiap ada pembaruan dokumen Firestore, *Stream* tersebut kembali memancarkan *List*. Setelah *List* ID didapatkan, aplikasi menggunakan Future.wait() untuk mengeksekusi request getMovieDetails(id) ke TMDb API secara massal demi melengkapi data gambar poster dan nama film, karena Firestore secara arsitektural didesain hanya merekam ID untuk mencegah duplikasi masif.
- **Screenshot Halaman My List**



Menampilkan tulisan "Daftar Saya", dengan 2 tab ("Wishlist" & "Watched"). Isi halamannya berupa deretan poster film vertikal berbentuk grid padat.

5.7 Manajemen Akun (Profile Screen)

- Fungsi Halaman: Modul akhir yang menangani kredensial dan preferensi visual pengguna.
- Alur Pembaruan Gambar:
 1. Pengguna menekan tombol pena di atas foto (Edit Foto).
 2. ImagePicker dipanggil dan sistem operasi host menyajikan dialog kamera/galeri.
 3. Setelah pengguna memilih berkas .jpg/.png, berkas lokal diumpankan ke fungsi upload ImgbbService.
 4. URL final dari respons HTTP ImgBB dipotong, dan FirebaseService memanggil `doc(uid).update({'profilePic': finalUrl})`. UI di-setState ulang.
- Screenshot Halaman Profil



Menampilkan foto profil bundar beserta ikon kamera, nama dan email, disusul kotak hijau/merah bertuliskan statistik "Total Watched" dan "Total Ulasan", lalu isian form "Ubah Nama" dan "Ganti Password", serta ditutup tombol merah "Logout" di bawah.

6. Prosedur Instalasi, Pembangunan (Build), dan Deployment

Proyek ini memerlukan sistem operasi pendukung untuk kompilasi kode native (Android Studio/Xcode).

6.1 Persiapan Lingkungan Pembangunan

1. Instalasi Flutter SDK: Pasang Flutter 3.11+ dan lakukan verifikasi instalasi menyeluruh via terminal:

```
flutter doctor -v
```

Pastikan tidak ada peringatan (silang merah) di sektor *Android toolchain*.

6.2 Tahapan Mode Pengembangan (Development)

1. Lakukan *Clone* (Salin) repositori dari platform Git ke penyimpanan lokal komputer.
2. Unduh semua paket dependensi yang dideklarasikan pada *pubspec.yaml*:

```
flutter pub get
```

3. (Opsional namun disarankan) Lakukan *Quality Control* statis untuk memastikan integritas koding:

```
flutter analyze
```

(Aplikasi ini dikembangkan hingga bebas *error* dan *warning*, 0 *issues* kritis tersisa).

4. Menyalakan aplikasi (baik melalui *emulator* Android Virtual Device atau penyambungan USB Kabel ke unit fisik):

```
flutter run
```

6.3 Kompilasi Sistem Produksi (APK Android Build)

Penciptaan modul instalasi siap disebarluaskan untuk platform Android dilakukan melalui mode rilis (`--release`). Proses ini akan mengaktifkan *Dart Ahead-of-Time* (AOT) compilation, *Tree-Shaking*, dan peniadaan log debugPrint.

```
flutter build apk --release
```

- Proses kompilasi awal mungkin membutuhkan waktu hingga puluhan menit karena Gradle diharuskan mengunduh kerangka kerja Android, mendamaikan pustaka Java/Kotlin, serta melakukan pengepakan final ke standar DEX.
- Titik henti (output) aplikasi akan diletakkan oleh Gradle di lintasan hierarki berikut: `build/app/outputs/flutter-apk/app-release.apk`
- Berkas `app-release.apk` ini langsung bisa disalin dan dibuka (pasang) secara bebas di perangkat berbasis Android manapun.

7. Skenario Pengujian Kualitas (UAT Checklist)

Daftar cek (User Acceptance Testing) berikut membuktikan bahwa fungsionalitas utama aplikasi telah diuji dan berjalan sebagaimana mestinya tanpa ada kendala (bug).

No.	Modul/Halaman	Skenario Pengujian (Aksi)	Hasil yang Diharapkan	Status
1	Autentikasi	Mendaftarkan akun menggunakan email baru	Akun berhasil dibuat dan pengguna diarahkan ke halaman Dashboard	√
2	Autentikasi	Login menggunakan password yang salah	Sistem menampilkan SnackBar berisi pesan kesalahan validasi dari Firebase Authentication	√
3	Dashboard	Menggulir (scroll) halaman hingga bagian paling bawah	Sistem secara otomatis memuat 20 film tambahan menggunakan fitur Infinite Scroll	√
4	Pencarian	Memasukkan kata kunci pencarian (misalnya: <i>Batman</i>)	Sistem menampilkan hasil pencarian yang sesuai berdasarkan data dari TMDB API	√
5	Detail Film	Menekan tombol Trailer YouTube	Sistem membuka aplikasi YouTube atau browser untuk memutar trailer (sesuai keamanan Android 11+)	√
6	Detail Film	Membuka detail film yang tidak memiliki sinopsis berbahasa Indonesia	Sistem secara otomatis menampilkan sinopsis berbahasa Inggris sebagai alternatif	√
7	Koleksi Saya	Menekan tombol + Wishlist	Status tombol berubah menjadi Tersimpan, dan data film berhasil disimpan ke Firebase	√
8	Koleksi Saya	Membuka halaman Daftar Saya (My List)	Sistem menampilkan daftar poster film yang telah ditambahkan ke wishlist pengguna	√
9	Ulasan	Menambahkan komentar dan memberikan rating pada film	Ulasan berhasil disimpan dan langsung ditampilkan pada daftar komentar	√
10	Profil Pengguna	Mengubah foto profil kemudian menyimpan perubahan	Foto profil berhasil diperbarui dan perubahan diterapkan secara sinkron pada ulasan yang pernah dibuat	√
11	Profil Pengguna	Mengubah nama profil kemudian menyimpan perubahan	Nama profil berhasil diperbarui dan tersinkronisasi pada seluruh ulasan yang telah dibuat	√

8. Analisis dan Kesimpulan Teknis

8.1 Evaluasi Kelebihan Sistem (Pros)

- Toleransi Kegagalan Data (Data Fallback Protocol): Program dirancang antisipatif (*defensive programming*). Contoh nyatanya adalah sistem perbaikan kecacatan TMDB API pada data sinopsis berbahasa Indonesia. Aplikasi mampu melakukan pengikatan data dari API bahasa Inggris sebagai pengganti otomatis apabila data terjemahan kosong, menghindari rusaknya layout (layar kosongan).
- Efisiensi Manajemen Memori (Infinite Scroll): Penerapan struktur parameter `&page=` yang dinamis dalam *API Service* serta sinkronisasi *ScrollController* menjauhkan aplikasi dari risiko kehabisan memori atau kelebihan muatan *CPU load*.
- Integritas Penulisan Kode: Tidak ada residu paket yang memboroskan APK (penghapusan `device_preview` dari kompilasi produksi), dan semua pencatatan konsol (`print`) telah dienkapsulasi menggunakan standar pelepasan memori Flutter (`debugPrint`).
- Kepatuhan Keamanan OS Modern: Aplikasi mengadopsi standar keamanan intent Android 11+ (API 30) dengan secara eksplisit mendefinisikan tag `<queries>` untuk skema `http/https` pada `AndroidManifest.xml`. Ini menjamin peluncuran URL eksternal (Trailer YouTube) dapat tereksekusi mulus tanpa diblokir oleh protokol privasi sistem operasi.

8.2 Analisis Kekurangan Arsitektur Saat Ini (Cons)

- Bottleneck Eksekusi Pemanggilan API pada List: Halaman *My List Screen* yang sangat ekstensif memanggil struktur interogasi siklus *for-loop* API independen terhadap `getMovieDetails` untuk masing-masing item ID di Firebase. Jika pengguna menaruh 500 film di *Wishlist*, aplikasi akan membebani TMDB API dengan 500 koneksi `get` sekaligus, berpotensi memicu *Rate Limit Block* jaringan atau *Timeout*.
- Struktur Manajemen State Terikat GUI: Logika komunikasi dengan *FirestoreService* masih diimpor dan diinstansiasi secara manual (`setState`) pada UI Screen (*DetailScreen*, *ProfileScreen*). Ini bertentangan dengan kaidah *Clean Architecture* dan membuat proyek rentan bila fungsionalitas membesar.

8.3 Saran Pengembangan Berkelanjutan

1. Pengadopsian Pustaka Pengaturan *State* Global: Refaktor arsitektur dari *StatefulWidget* murni ke paradigma BLoC (*Business Logic Component*) atau Riverpod. Ini akan memisahkan UI sepenuhnya dari pengambil keputusan data (*Data Layer*).
2. Sistem Singgahan Luring (*Offline Caching Layer*): Memasukkan librari Hive atau SQLite untuk menyimpan data detail JSON sementara. Ini meniadakan pembaharuan muatan pada halaman *My List* ketika berstatus tiada internet, serta menyelamatkan kuota akses TMDB API.
3. Sentralisasi Penyimpanan Aset Pribadi: Integrasi fitur pengunggahan foto profil ke Firebase Storage (arsitektur asli Google), ketimbang menitipkan aset berharga pengguna pada server API eksternal tak berbayar yang tak terkontrol dan memiliki batasan tingkat transfer lalu lintas akses (ImgBB).